

```
---
title: "Lab 1 Forecasting with ARIMA and ETS models"
author: 'Team 3: Ben Mahklouf, Tina Tao, and Emma Christman'
output:
  html_document:
    code-folding: true
    toc: true
    toc_float: true
  pdf_document:
    toc: true
---
```

```
```{r include=FALSE}
library(tidyverse)
options(dplyr.summarise.inform = FALSE)
knitr::opts_chunk$set(warning = FALSE)
library(forecast)
```
```

#Initial Plan and What Actually Happened

We will compare the accuracy of ARIMA models in yearly vs. monthly data, answering the question: Is the accuracy of monthly or yearly data better? To do this we will look only at the Washington data. To begin, we will test the stationarity of the yearly data and the monthly data.

If either are not stationary, we will use differencing to make them stationary for the purposes of this analysis. After the data are stationary, we will fit an ARIMA and ETS model to the monthly data using AutoARIMA on the earliest 80% of the data.

This will allow use to compare the accuracy of the ARIMA model in predicting the last 20% of the data to the actual data. After this is done, we will do the same to the yearly data. Then, we can use the accuracy function to compare the two methods.

We loaded the data and wrangled it, filtering it to only include the Washington data. This allowed us to compare between both the yearly and monthly data. From there we used the `auto.arima()` function, which auto-checked for stationarity for us. We needed to make the yearly data stationary by differencing it once. Then, we fit both ETS and arima models to both the monthly and yearly catch data from Washington. After fitting the models, we concluded that the monthly data forecast was more accurate than the yearly data forecast.`

#Question Addressed

- 1) Is an ARIMA model and/or an ETS model adequate for using monthly data and yearly data?
- 2) Is the accuracy of forecasting with monthly data or yearly data "better"?

```

# Step1, load data and filter to Washington with basic plots

```{r}
Seperate the monthly and yearly data
library(atsalibrary) # This is ALL the data

This is the monthly data
chinook.month <- chinook.month %>%
 mutate(Date = as.Date(paste(Year, Month, "01", sep = "-"), format = "%Y-
%b-%d"))

chinook.month <- chinook.month %>%
 filter(State %in% c("WA"))

chinook.year <- chinook.year

chinook.year_filtered <- chinook.year %>%
 filter(State == "Washington")

Quick plot of year

ggplot(chinook.year_filtered, aes(x = Year, y = log.metric.tons)) +
 geom_line() +
 facet_wrap(~ State, ncol = 1) + # 3 panels vertically
 labs(title = "Monthly Chinook Salmon Landings",
 x = "Date",
 y = "Log(Metric Tons)") +
 theme_minimal()

Quick plot of month

ggplot(chinook.month, aes(x = Date, y = log.metric.tons)) +
 geom_line() +
 facet_wrap(~ State, ncol = 1) + # 3 panels vertically
 labs(title = "Monthly Chinook Salmon Landings",
 x = "Date",
 y = "Log(Metric Tons)") +
 theme_minimal()

```

# Step2, check stationarity and differencing

```{r}

yearly <- chinook.year_filtered %>%
 arrange(Year) %>%
 pull(log.metric.tons)

```

```
yearly_ts <- ts(yearly, start = 1950, end = 2016, frequency = 1)
```

```
monthly<- chinook.month %>%
 #change log metric tons inf to NA
 mutate(log.metric.tons = ifelse(log.metric.tons == Inf, NA,
log.metric.tons)) %>%
 mutate(log.metric.tons = ifelse(log.metric.tons == -Inf, NA,
log.metric.tons)) %>%
 filter(!is.na(log.metric.tons))
```

```
monthly <- monthly %>%
 arrange(Date) %>%
 pull(log.metric.tons)
```

```
Are there NAs in monthly
remove NA
```

```
monthly_ts <- ts(monthly, start = c(1990,1), frequency = 12)
```

```
#####
```

```
Check the need to difference the data
```

```
adf.test(yearly_ts)
```

```
Augmented Dickey-Fuller Test
```

```

data: yearly_ts
Dickey-Fuller = -2.5627, Lag order = 4, p-value = 0.3467
alternative hypothesis: stationary
```

```
Not stationary, p value greater than .01
```

```
adf.test(monthly_ts)
```

```
Augmented Dickey-Fuller Test
```

```

data: monthly_ts
Dickey-Fuller = -9.6198, Lag order = 7, p-value = 0.01
alternative hypothesis: stationary
```

```
These data ARE Stationary
```

```
#####
```

```
We can run the ndiff to see how many differences we would need to get to
```

```

stationarity

ndiffs(yearly_ts)

[1], we need to difference ONCE, we expect the middle number in autoarima
to be 1

...

Step 3, explore correlation and autocorrelation using ACF and PACF

```{r}
acf(yearly_ts, lag.max = 20, main = "ACF of Yearly Chinook Salmon Landings
in Washington")
# The ACF plot shows a tails off structure
pacf(yearly_ts, lag.max = 20, main = "PACF of Yearly Chinook Salmon
Landings in Washington")
# The PACF plot show a cuts off structure after lag=1
# These are AR(1) model characteristics
# Combining with results from the above stationary test, we expect an
ARIMA(1,1,0) model to fit the yearly Chinook Salmon Landings in Washington

acf(monthly_ts, lag.max = 36, main = "ACF of Monthly Chinook Salmon
Landings in Washington")
# We observe a clear deterministic seasonal pattern and expect a big
negative coefficient for AR component
pacf(monthly_ts, lag.max = 36, main = "PACF of Monthly Chinook Salmon
Landings in Washington")
# Several correlation patterns are detected and there is a strong
correlation with lag=2
# Choosing the optimal p and q value may need to be done by comparing AIC
and BIC values, which is expected to be conducted in auto.arima function
```

Step 4, fit ARIMA and ETS models using created training and test sets
on the monthly and yearly data

```{r}

# Create training and test sets (80% training, 20% test)
yearly_train_end <- floor(0.8 * length(yearly_ts))
yearly_train <- window(yearly_ts, end = time(yearly_ts)[yearly_train_end])
yearly_test <- window(yearly_ts, start = time(yearly_ts)[yearly_train_end +
1])

monthly_train_end <- floor(0.8 * length(monthly_ts))
monthly_train <- window(monthly_ts, end = time(monthly_ts)
[monthly_train_end])
monthly_test <- window(monthly_ts, start = time(monthly_ts)
[monthly_train_end + 1])

# Delete monthly_train_end

```

```

rm(monthly_train_end)
# Delete yearly_train_end
rm(yearly_train_end)

# Fit ARIMA model to training data
yearly_arma <- auto.arima(yearly_train)
# Series: yearly_train
# The model result suggests an random walk ARIMA(0,1,0) instead of
ARIMA(1,1,0), because the AR(1) coefficient might be small or not
statistically significant
#
# sigma^2 = 0.04824: log likelihood = 5.04
# AIC=-8.07 AICc=-7.99 BIC=-6.12

monthly_arma <- auto.arima(monthly_train)

# Series: monthly_train
# ARIMA(2,0,2) with non-zero mean
#
# Coefficients:
#          ar1      ar2      ma1      ma2      mean
#          1.6522 -0.9335 -1.1833  0.4651  3.2705
# s.e.      0.0423  0.0444  0.1831  0.1728  0.0830
#
# sigma^2 = 2.048: log likelihood = -526.49
# AIC=1064.98 AICc=1065.27 BIC=1087.14

# Fit ETS models
yearly_ets <- ets(yearly_train)

# ETS(A,N,N)
#
# Call:
# ets(y = yearly_train)
#
# Smoothing parameters:
#   alpha = 0.9999
#
# Initial states:
#   l = 8.2942
#
# sigma: 0.2218
#
#      AIC      AICc      BIC
# 54.74317 55.23297 60.65405

monthly_ets <- ets(monthly_train)

# ETS(A,N,N)

```

```

#
# Call:
# ets(y = monthly_train)
#
# Smoothing parameters:
#   alpha = 0.9999
#
# Initial states:
#   l = 3.2657
#
# sigma: 1.8451
#
#      AIC      AICc      BIC
# 2058.871 2058.953 2069.952
#
...

### Step 5, model performance residual diagnostics
```{r}
checkresiduals(yearly_arma)
ACF of residuals shows no significant autocorrelation and looks like a
normal distribution, indicating a very good fit
checkresiduals(monthly_arma)
ACF of residuals have two autocorrelation spikes but could be random
weird thing and looks like a normal distribution, indicating a relative
good fit
checkresiduals(yearly_ets)
ACF of residuals shows no significant autocorrelation and looks like a
normal distribution, indicating a good fit
checkresiduals(monthly_ets)
ACF of residuals shows several significant autocorrelations though looks
like a normal distribution, indicating a poor fit

In general, the ARIMA models seem to fit better than the ETS models for
both yearly and monthly data.
```

### Step 6, forecast
```{r}

Generate forecasts for the test period
yearly_arma_forecast <- forecast(yearly_arma, h = length(yearly_test))
yearly_ets_forecast <- forecast(yearly_ets, h = length(yearly_test))

monthly_arma_forecast <- forecast(monthly_arma, h = length(monthly_test))
monthly_ets_forecast <- forecast(monthly_ets, h = length(monthly_test))

Plot forecasts
plot1<-autoplot(yearly_arma_forecast)
plot2<-autoplot(yearly_ets_forecast)

```

```

plot3<-autoplot(monthly_arima_forecast)
plot4<-autoplot(monthly_ets_forecast)

Put into a 4 plot panel
library(gridExtra)
library(grid)
library(ggplot2)

grid.arrange(plot1, plot2, plot3, plot4, ncol = 2)

...

Accuracy metrics
```{r}
# Evaluate forecast accuracy
yearly_arima_accuracy <- accuracy(yearly_arima_forecast, yearly_test)

#           ME      RMSE      MAE      MPE      MAPE      MASE
ACF1 Theil's U
# Training set -0.01046748 0.2175570 0.1732156 -0.1712576 2.225795
0.9820193 0.0292464      NA
# Test set      -0.07686666 0.2753066 0.2363580 -1.1236585 3.107135
1.3399954 0.5697271 1.139735

yearly_ets_accuracy <- accuracy(yearly_ets_forecast, yearly_test)

#           ME      RMSE      MAE      MPE      MAPE      MASE
ACF1
# Training set -0.01062423 0.2175548 0.1730609 -0.1731526 2.223930
0.9811418 0.02860653
# Test set      -0.07683952 0.2752990 0.2363503 -1.1233035 3.107024
1.3399514 0.56972710

monthly_arima_accuracy <- accuracy(monthly_arima_forecast, monthly_test)

#           ME      RMSE      MAE      MPE      MAPE      MASE
ACF1 Theil's U
# Training set -0.0620450 1.314323 0.9435214 -Inf   Inf 0.8521606
-0.001970859      NA
# Test set      -0.9640865 3.266402 2.5277314 -Inf   Inf 2.2829721
0.737652808      0

monthly_ets_accuracy <- accuracy(monthly_ets_forecast, monthly_test)

#           ME      RMSE      MAE      MPE      MAPE      MASE      ACF1
Theil's U

```

```
# Training set  0.0008512003 1.653080 1.088517 -Inf  Inf 0.983116 0.3694314
NA
# Test set      -1.8938374286 3.914736 2.998623 -Inf  Inf 2.708267 0.7424152
0
...

```

Results and Discussion

The accuracy metrics show that for yearly data, ARIMA and ETS models performed nearly the same with a MAPE value of about 3.1%, indicating reasonably accurate forecasts. For the monthly data, the ARIMA model outperformed ETS with lower RMSE (3.27 vs 3.91) and MAE (2.53 vs 3.00)

Visually, the plots suggest better monthly performance but the numerical metrics indicate higher absolute errors for monthly forecasts compared to yearly ones. It would be interesting to compare how each of these predictions perform on test data from the other dataset (i.e. do predictions made from monthly data perform better for yearly data). Similarly, it would be interesting to compare the results here to that of the data from California, which is made complicated by the different range of time series values. We questioned the legitimacy of our methods for fitting an ARIMA to the yearly data, as the model suggests that the predicted catch for the next years is equal to the last year's catch. This held true with some back-of-the envelope calculations using the same methods on the California and Oregon data. Specifically, we question the fact that the forecast plot visually shows no variation that would typically be associated with a random walk, rather forecasting a straight line with no variation.

##Team Member Contributions

BM, EC, and TT all contributed code, question design, and analysis to the final work presented here. EC wrote details about the initial plan and questions we addressed, and how the plan was carried out, as well as added details to the discussion section. TT added details about checking the residuals and the ACF/PACF plots, and wrote details about the results and discussion. BM wrote and organized final code and added details about Accuracy results.